
A06	DAO & Database Access 3-Layer Architecture	Algorithmic Thinking ALD (MTD.ba VZ) SS26
Submission via Moodle		24 Points

In this assignment, you implement a full 3-layer architecture for a small persistence scenario. The architectural separation is mandatory for everyone, but you may choose your data-access style: **classic JDBC DAO** or **Spring Data JPA repositories**.

Submission Requirements:

- Read the `Organizational Guidelines` on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and explanations
 - Include all source files in a ZIP archive
 - Use the naming convention: `ALD_A06_LASTNAME.pdf` and `ALD_A06_LASTNAME.zip`
-

Exercise 1 (12 points) Exercise 1 — Architecture and Core CRUD

Build a small application in **three layers**:

- `presentation/`: user interaction (console menu or scripted runner)
- `service/`: validation and orchestration
- `dataaccess/`: persistence code (JDBC DAO *or* Spring Data JPA)

Implement the following two domain entities and their relationship:

Entity	Field	Details
Genre	<code>id</code>	auto-increment primary key
	<code>name</code>	unique, not null
Game	<code>appId</code>	primary key (from CSV)
	<code>name</code>	not null
	<code>releaseDate</code>	
	<code>price</code>	
	<code>reviewScore</code>	

Relationship: M:N — a Game belongs to multiple Genres; a Genre covers many Games. In the database, realise this via a junction table `game_genre(game_id, genre_id)`.

Part A — Structure & Contracts (3 points)

Define clear interfaces/contracts in `dataaccess/` and wire everything through service into presentation.

Part B — CRUD Implementation (6 points)

Implement create/read/update/delete for both entities. For Game, saving must also insert the genre links; deleting must remove them first (or use cascade).

Part C — Service Validation (3 points)

In the service layer, enforce: blank name exception, negative price exception, empty genre list exception.

Exercise 2 (12 points) Exercise 2 — Query Use-Cases, Testing, and Demo

Use your architecture for realistic query workflows.

Part A — Query Operations (4 points)

Implement the following two filtered reads in your DAO and expose them through your service:

- `findByGenre(int genreId): List<Game>` — returns all games that belong to the given genre
- `findByPriceRange(double min, double max): List<Game>` — returns all games with `price >= min` and `price <= max`

Part B — Data Import/Seed (3 points)

Import data from the provided file `steam_games.csv`. Import *all* rows that are valid for your data model:

- parseable `steam_appid` (integer) and `price_initial` (USD) (non-negative double)

- non-blank name
- `release_date` that is not blank and not `Not Released`
- at least one genre in the `genres` column (which is a Python-style list: `['Action', 'RPG']`)

Use only these six columns. Parse *all* genres per row and upsert each into the `genre` table before inserting into `game_genre`. Silently skip invalid rows.

Part C — Tests (3 points)

Add JUnit 5 tests for `findByGenre`, `findByPriceRange`, and the core CRUD operations. Show `mvn test` output in your PDF.

Part D — End-to-End Demo (2 points)

Show a short runnable demo flow from presentation layer through service to data access. Include console output in your PDF.

Track-Specific Notes

- **JDBC DAO track:** Use `PreparedStatement` for all user-influenced queries and keep SQL in `dataaccess/` only.
- **Spring Data JPA track:** Use `@Entity` + `@ManyToMany` and extend `JpaRepository`. Mark write operations in service with `@Transactional`.
- **Both tracks:** The grading focus is architecture quality and correctness, not framework preference.